| Practical:11 | Write a program for Floyd-Warshal algorithm. |
|---|---|
| __/__ /2026 | |

**Objective**

- To implement the **Floyd-Warshall algorithm** to find the All-Pairs Shortest Paths in a weighted directed graph (allowing negative edge weights, provided there are no negative weight cycles).
- To understand the dynamic programming technique of using intermediate vertices to iteratively improve path costs.
- To analyze the time and space complexity of the Floyd-Warshall algorithm.

**Program**

```
# Floyd-Warshall Algorithm for All-Pairs Shortest Path
INF = float('inf')


def floyd_warshall(graph):
    V = len(graph)
    # Initialize distance matrix with graph values
    dist = [row[:] for row in graph]

    # Dynamic Programming: k is the intermediate vertex
    for k in range(V):
        for i in range(V):
            for j in range(V):
                # If vertex k is on the shortest path from i to j, update dist[i][j]
                if dist[i][k] != INF and dist[k][j] != INF:
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

    # Check for negative weight cycles
    for i in range(V):
        if dist[i][i] < 0:
            raise ValueError("Graph contains a negative weight cycle!")

    return dist
```

```
# Driver Code
if __name__ == "__main__":
    # Graph represented as an adjacency matrix (V = 4)
    # INF represents no direct edge between vertices
    graph = [
        [0,   3,   INF, 7],
        [8,   0,   2,   INF],
        [5,   INF, 0,   1],
        [2,   INF, INF, 0]
    ]

    V = len(graph)

    print("FLOYD-WARSHALL ALGORITHM (All-Pairs Shortest Path)")
    print("\nInitial Distance Matrix (Direct Edges):")
    for row in graph:
        print([r if r != INF else "INF" for r in row])

    # Compute shortest distance matrix
    shortest_dist = floyd_warshall(graph)

    print("\nFinal Shortest Distance Matrix:")
    for row in shortest_dist:
        print([r if r != INF else "INF" for r in row])
```

**Output**

```
FLOYD-WARSHALL ALGORITHM (All-Pairs Shortest Path)

Initial Distance Matrix (Direct Edges):
[0, 3, 'INF', 7]
[8, 0, 2, 'INF']
[5, 'INF', 0, 1]
[2, 'INF', 'INF', 0]

Final Shortest Distance Matrix:
[0, 3, 5, 6]
[5, 0, 2, 3]
[3, 6, 0, 1]
[2, 5, 7, 0]
```

**Analysis**

**a) Problem Analysis**

The **All-Pairs Shortest Path (APSP)** problem requires finding the minimum path cost between every pair of vertices $(i, j)$ in a directed or undirected graph. Unlike single-source shortest path algorithms like Dijkstra's or Bellman-Ford (which start from a single source node), Floyd-Warshall computes shortest paths between all pairs simultaneously.

**b) Dynamic Programming Approach**

Floyd-Warshall uses a **Dynamic Programming** strategy based on intermediate vertices:

- Let $dist[i][j]^{(k)}$ be the shortest distance from vertex $i$ to vertex $j$ considering only vertices $\{0,1,\dots,k-1\}$ as potential intermediate nodes on the path.
- At step $k$, we decide whether routing through vertex $k$ yields a shorter path from $i$ to $j$ than the best path found so far:

$$\text{dist}[i][j] = \min(\text{dist}[i][j],\ \text{dist}[i][k] + \text{dist}[k][j])$$

**c) Example Trace**

For the 4-vertex graph ($V = 0,1,2,3$):

1. **Initial Matrix ($k = -1$):**
   - $D^{(0)} = \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{bmatrix}$
2. **After $k = 0$ (using vertex 0 as intermediate):**
   - Path $3 \to 0 \to 1$: $dist[3][1] = \min(\infty, dist[3][0] + dist[0][1]) = \min(\infty, 2 + 3) = 5$

- Path $3 \rightarrow 0 \rightarrow 3$: $dist[3][3] = \min(0, 2 + 7) = 0$

3. **After $k = 1$ (using vertex 1 as intermediate):**
   - Path $0 \rightarrow 1 \rightarrow 2$: $dist[0][2] = \min(\infty, 3 + 2) = 5$
4. **After $k = 2$ (using vertex 2 as intermediate):**
   - Path $1 \rightarrow 2 \rightarrow 3$: $dist[1][3] = \min(\infty, 2 + 1) = 3$
   - Path $0 \rightarrow 2 \rightarrow 3$: $dist[0][3] = \min(7, 5 + 1) = 6$
5. **After $k = 3$ (using vertex 3 as intermediate):**
   - All entries evaluated. The final matrix gives the absolute shortest distance between every pair $(i, j)$.

**d) Algorithm Steps**

1. Create a distance matrix dist initialized to the graph's weight matrix. Set dist[i][i] = 0 and non-existent edges to INF.
2. Run three nested loops:
   - Outer loop $k$ selects the candidate intermediate vertex (0 to $V - 1$).
   - Middle loop $i$ selects the source vertex (0 to $V - 1$).
   - Inner loop $j$ selects the destination vertex (0 to $V - 1$).
3. Update dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]).
4. If any diagonal element dist[i][i] < 0, a **negative cycle** is present in the graph.

**e) Time Complexity**

- Three nested loops each run $V$ times.
- The inner operation takes $O(1)$ constant time.
- **Overall Time Complexity:** $O(V^3)$ (regardless of graph density).

**f) Space Complexity**

- Requires a $V \times V$ matrix to store distance values.
- **Overall Space Complexity:** $O(V^2)$ auxiliary space.

**Conclusion**

The Floyd-Warshall algorithm was successfully implemented in Python using Dynamic Programming. It computes all-pairs shortest paths in $O(V^3)$ time and $O(V^2)$ space, making it simple to implement and optimal for dense graphs or small to medium vertex counts ($V \leq 500$).